

Phase 1 작업 명세서 — 정확성 버그 4건

재개 지점. 상위 계획: [Refactor_Plan.md](#) §Phase 1 / §6 다음 세션 가이드 성격: 수치 무관 · 구조 변경 없음 · 잠재 크래시/영구차단 리스크 제거만이 목적. Phase 2(레지스트리) 이전에 "타이밍 의존 UB"를 먼저 걷어내 리팩터 기준선을 깨끗하게 만든다.

작업 원칙

- 각 버그 = 독립 커밋 = 롤백 지점. 4건을 한 커밋에 몰지 말 것.
- 동작 보존이 기본. 정상 경로의 결과는 그대로여야 하고, 바뀌는 건 "실패/경계 경로의 안전성"뿐.
- 커밋 컨벤션: [리소스 Phase 1 — <버그 요약>](#)
- 검증: 에디터(또는 배치) 컴파일 → 디버그 창([Tools > Resource Debug](#))으로 썬 왕복 → 팝업 열고 닫기. 수치는 안 움직여야 정상.
- 함정(메모리 기록): 유저 Unity 에디터가 열려 있으면 배치 컴파일 불가. [BaseCharacter.cs](#) 류 열린 파일은 저장버퍼가 Edit을 덮어쓰 — 편집할 파일은 IDE에서 닫고 작업.

대상 파일 요약

버그	파일	핵심 위치
① NoCache use-after-release	ResourceManager.cs / DataManager.cs	LoadAsyncNoCache L141~171 / LoadAndCacheJsonAsync L73-L82
② 캐시 히트 승격 누락	ResourceManager.cs	LoadAsync<T>(string,...) 캐시 히트 L65~69
③ 해제 순서 + 이중 Clear	BaseScene.cs / LoadingScene.cs / SceneManagerEx.cs	BaseScene.Clear() L29
④ _isLoadingPopup 미복구	UIManager.cs	ShowPopupUIAsync L118

경로 접두사 생략: [Assets/Scripts/Managers/Core/](#) (③의 [BaseScene/LoadingScene](#)은 [Assets/Scripts/Scenes/...](#)).

버그 ① — [LoadAsyncNoCache](#) use-after-release (P2)

현상 / 근거

[LoadAsyncNoCache<T>](#) 는 핸들을 [Release](#) 한 뒤 그 result를 반환한다.

```
// ResourceManager.cs L159~163
if (handle.Status == AsyncOperationStatus.Succeeded && result != null)
{
    Addressables.Release(handle); // ← 먼저 해제
    return result;               // ← 해제된 에셋을 반환
}
```

호출부는 반환 후에 에셋을 쓴다:

```
// DataManager.cs L73-L82
TextAsset textAsset = await Managers.Resource.LoadAsyncNoCache<TextAsset>(addressableKey);
...
Loader loader = JsonUtility.FromJson<Loader>(textAsset.text); // ← 해제 뒤 .text 접근
```

[Release](#) 로 refcount가 0이 되면 번들 언로드로 [TextAsset](#) 이 파괴될 수 있고, 그 뒤 [.text](#) 접근은 use-after-release. 현재는 Addressables의 언로드 지연 덕에 "우연히" 동작 — 타이밍 의존 UB.

원인

"로드 → 즉시 해제 → 반환"이라는 계약 자체가 틀렸다. 소비가 끝나기 전에 소유권을 놓는다.

수정 방침 (권장안)

소비를 메서드 안으로 끌어와서, 핸들이 살아있는 동안 값 복사 → 그 다음 해제. TextAsset의 유일한 소비는 `.text` (string)이므로 문자열 전용 메서드로 특화한다.

`ResourceManager.cs` — 신규 메서드:

```
// NoCache 텍스트 로드: 핸들이 살아있는 동안 문자열을 복사한 뒤 해제 (use-after-release 제거)
public async UniTask<string> LoadTextNoCacheAsync(string key)
{
    if (string.IsNullOrEmpty(key)) return null;

    var handle = Addressables.LoadAssetAsync<TextAsset>(key);
    try
    {
        var asset = await handle.ToUniTask();
        if (handle.Status == AsyncOperationStatus.Succeeded && asset != null)
            return asset.text; // string은 값 복사 → 핸들 해제해도 안전

        GameLog.LogError($"[ResourceManager] NoCache Text Load Failed: {key}");
        return null;
    }
    catch (Exception e)
    {
        GameLog.LogError($"[ResourceManager] NoCache Text Load Exception: {key} / {e.Message}");
        return null;
    }
    finally
    {
        if (handle.IsValid()) Addressables.Release(handle); // 복사 끝난 뒤 해제 (finally로 예외경로도 커버)
    }
}
```

`DataManager.cs LoadAndCacheJsonAsync` — 호출부 전환:

```
string json = await Managers.Resource.LoadTextNoCacheAsync(addressableKey);
if (json == null)
{
    GameLog.LogError($"[DataManager] JSON 로드 실패: '{addressableKey}' (Type: {typeof(TValue).Name})");
    return;
}
Loader loader = JsonUtility.FromJson<Loader>(json);
_dataDicts[typeof(TValue)] = loader.MakeDict();
```

기존 `LoadAsyncNoCache<T>` 는 다른 호출부가 없으므로(grep 확인: DataManager 1곳뿐) 삭제한다. 삭제 시 컴파일 에러가 나면 남은 호출부를 같이 전환.

대안(제네릭 유지): 시그니처를 `UniTask<(T asset, AsyncOperationHandle handle)>` 로 바꿔 호출부가 사용 후 직접 `Release` . — 호출부가 해제를 "기억"해야 하므로 이 리팩터의 취지(기억을 구조로 대체)와 정반대. 권장 안(문자열 특화)을 택할 것.

검증

- 게임 부팅 시 데이터 로드 로그에 실패 없음(모든 JSON 정상 파싱). `_dataDicts` 채워지는지 몇 종 확인.
- 존재하지 않는 키를 임시로 넣어 실패 경로가 로그만 남기고 크래시 없는지(선택).

버그 ② — 캐시 히트 시 global 승격 누락 (P1 일부)

현상 / 근거

`LoadAsync<T>(string key, bool isGlobal, ...)` 의 동기 캐시 히트 경로가 `isGlobal` 인자를 무시한다.

```
// ResourceManager.cs L65~69
if (_globalHandles.TryGetValue(key, out var gh) && gh.IsDone && gh.Result is T gResult)
    return UniTask.FromResult(gResult);

if (_sceneHandles.TryGetValue(key, out var sh) && sh.IsDone && sh.Result is T sResult)
    return UniTask.FromResult(sResult); // ← scene에 있으면 isGlobal:true여도 그냥 반환
```

키가 scene 버킷에 먼저 로드된 뒤, 누군가 같은 키를 `isGlobal:true` 로 요청하면 → scene 캐시가 히트해서 그대로 반환 → global로 승격 안 됨. 이후 썬 전환 때 `Clear()` 가 그 핸들을 release → global인 줄 알고 잡고 있던 호출부는 파괴된 에셋을 참조.

원인

수명 의도(isGlobal)가 캐시 조회 단계에서 소실. Phase 2에서 `(key,type) + refCount` 레지스트리로 근본 해결되지만, 지금은 승격 로직을 명시적으로 넣어 리스크를 제거한다.

수정 방침

scene 히트인데 `isGlobal:true` 요청이면 핸들을 scene→global로 재배치(승격) 후 반환. 핸들은 이미 완료 상태라 동기 디커플러 이동만으로 안전하다. 역방향(global 히트에 scene 요청)은 강등하지 않는다 — global은 더 긴 수명이라 그대로 두는 게 안전.

```
if (_globalHandles.TryGetValue(key, out var gh) && gh.IsDone && gh.Result is T gResult)
    return UniTask.FromResult(gResult);

if (_sceneHandles.TryGetValue(key, out var sh) && sh.IsDone && sh.Result is T sResult)
{
    if (isGlobal) // 수명 승격: 썬 Clear에 달려 파괴되지 않도록 global 버킷으로 이관
    {
        _sceneHandles.Remove(key);
        _globalHandles[key] = sh;
    }
    return UniTask.FromResult(sResult);
}
```

`LoadAsyncInternal` (L82~92)에도 타입 불일치 시 `wasGlobal` 승격 로직이 이미 있음 — 여기 추가하는 건 타입 일치 캐시 히트 경로의 누락분. 둘이 상호 배타적이라 충돌 없음.

검증

- 디버그 창에서 어떤 키가 scene→global로 이동하는 케이스가 실제로 있는지 확인(있으면 승격 로그 시점에 Scene 카운트 -1 / Global +1). 없어도 정상(방어적 수정).
- 정상 경로 회귀 없음: 썬 왕복 후 잔존 핸들 수가 기존과 동일.

버그 ③ — 썬 핸들 해제 순서 일원화 + 이중 Clear 제거 (P3)

현상 / 근거

썬 핸들이 두 번, 그것도 이른 시점에 해제된다.

- 이른 해제(위험): `SceneManagerEx.LoadSceneAsync` 가 아직 이전 썬이 화면에 떠 있는 상태에서 `CurrentScene.Clear()` 호출.

```
// SceneManagerEx.cs L70~86
if (CurrentScene != null)
    CurrentScene.Clear(); // → BaseScene.Clear() → Managers.Resource.Clear() (썬 핸들 전부 Release)
Managers.Clear();
...
SceneManager.LoadScene("Loading"); // ← 이 시점 전에 이미 텍스처/메시가 release됨 = 언로드 창(window)
```

```
// BaseScene.cs L27~31
public virtual void Clear()
{
    Managers.Resource.Clear(); // ← 문제의 이른 release
}
```

2. 이중 해제: Loading 씬 진입 후 `LoadProcessAsync` 가 또 `Resource.Clear()` .

```
// LoadingScene.cs L39
Managers.Resource.Clear();
```

`Managers.Clear()` 는 `SceneEx.Clear()` (빈 메서드) + `Field.Clear()` 뿐이라 Resource와 무관 — 즉 Resource 해제 지점은 위 2곳.

원인

씬 핸들 해제가 "이전 씬 렌더 중"에 일어나 화면에 떠 있는 오브젝트의 텍스처/메시가 파괴될 수 있는 창이 열린다. 해제 시점을 Loading 씬 진입 이후 한 곳으로 일원화해야 한다.

수정 방침

`BaseScene.Clear()` 에서 `Managers.Resource.Clear()` 한 줄을 제거한다. 그러면:

- `CurrentScene.Clear()` 는 이전 씬 정리(이벤트 구독 해제·사운드 정지 등 서브클래스 고유 로직은 유지)만 하고 핸들은 건드리지 않음 → 렌더 창 제거.
- 씬 핸들 실제 release는 `LoadingScene.LoadProcessAsync` (L39, Loading 씬이 이미 활성화)에서 단독 수행 → 이중 호출도 소멸.

```
// BaseScene.cs
public virtual void Clear()
{
    // 씬 핸들 해제는 Loading 씬 진입 후 LoadingScene.LoadProcessAsync에서 일원화 수행 (P3)
    // 여기서 Resource.Clear()를 하면 이전 씬이 아직 렌더 중이라 언로드 창이 열린다.
    //Managers.Resource.Clear();
}
```

서브클래스 확인(중요): `GameScene` / `SelectScene` / `NormalDungeonScene` / `BossDungeonScene` / `StartScene` 가 `Clear()` 를 override하고 `base.Clear()` 를 호출한다. base가 비어도 각자의 정리(이벤트 unsubscribe, `Sound.StopAll` , `Dungeon.ClearDungeonData` , `Resource.Destroy(_clearUI)` 등)는 그대로 실행되므로 문제 없음 — 이들은 핸들 release가 아니라 인스턴스/구독 정리라 이른 시점에 해도 무방. 별도 수정 불필요.

불변식 확인: 모든 씬 전환은 `SceneManagerEx.LoadSceneAsync` → Loading 씬 경로 하나뿐이므로, Loading 씬의 `Resource.Clear()` 가 반드시 실행된다 → 핸들 누수 없음. Loading 씬 직후 `AssertSceneHandlesCleared("LoadingScene Clear 직후")` (LoadingScene.cs L44)가 이미 이를 검증 중.

검증

- 씬 왕복(예: Select ↔ Game/Dungeon) 후 디버그 창에서 Scene 버킷 = 기존과 동일하게 비워짐. `[ResourceLeak]` assert 로그가 뜨지 않아야 함.
- 씬 전환 순간 화면 잔상/핑크 텍스처 없이 매끄러운지 육안 확인.
- `[TransitionMetric]` 로그: 전환 피크가 예전과 유사(이 수정으로 크게 바뀌면 안 됨 — 안전성 수정이지 메모리 수정 아님).

버그 ④ — `_isLoadingPopup` 미복구로 팝업 영구 차단 (0.5a 발견)

현상 / 근거

```
// UIManager.cs L107~118
public async UniTask<T> ShowPopupUIAsync<T>(string addressableKey = null) where T : UI_PopUp
{
    if (_isLoadingPopup) return null;
    _isLoadingPopup = true;
    ...
    GameObject prefab = await Managers.Resource.LoadAsync<GameObject>(addressableKey, isGlobal: true);
    if (prefab == null) return null; // ← _isLoadingPopup = true 인 채로 조기 반환!
    ...
    _isLoadingPopup = false; // L140 — 성공 경로에서만 복구됨
    return popup;
}
```

prefab 로드가 실패(오타 키·번들 누락·로드 취소)하면 `_isLoadingPopup` 이 `true` 로 박제된다. `IsPopupOpen` (L25 = `_popupStack.Count>0 || _isLoadingPopup`)이 계속 `true` → 이후 모든 팝업 열기가 L109에서 영구 차단, ESC 메뉴도 안 열림.

원인

플래그를 여러 return 경로에서 수동으로 되돌려야 하는 구조인데 실패 경로에서 빠뜨렸다. `try/finally` 로 복구를 구조화한다.

수정 방침

플래그 해제를 `finally` 로 옮겨 모든 반환/예외 경로에서 반드시 복구되게 한다. 성공 경로의 `_isLoadingPopup = false` (L140)는 제거하고 `finally`로 합친다.

```
public async UniTask<T> ShowPopupUIAsync<T>(string addressableKey = null) where T : UI_PopUp
{
    if (_isLoadingPopup) return null;
    _isLoadingPopup = true;
    try
    {
        if (string.IsNullOrEmpty(addressableKey))
            addressableKey = typeof(T).Name;

        ResourceMetrics.BeginUIOpen(addressableKey);

        GameObject prefab = await Managers.Resource.LoadAsync<GameObject>(addressableKey, isGlobal: true);
        if (prefab == null) return null; // 이제 finally가 플래그를 되돌려 줌

        GameObject go = Managers.Resource.Instantiate(prefab, CanvasPopup.transform);
        go.transform.SetParent(CanvasPopup.transform, false);
        go.SetActive(true);

        RectTransform rect = go.GetComponent<RectTransform>();
        rect.anchoredPosition = Vector2.zero;
        rect.localScale = Vector3.one;

        T popup = Util.GetOrAddComponent<T>(go);
        _popupStack.Push(popup);
        Managers.Input.PushContext(InputContext.UI);

        GameLog.Log($"팝업 스택 : {_popupStack.Count}");
        SetCanvas(go, true);

        ResourceMetrics.MarkUIShown(addressableKey);
        return popup;
    }
    finally
    {
        _isLoadingPopup = false; // 성공·실패·예외 전 경로 복구
    }
}
```

참고: 실패 시 `ResourceMetrics.BeginUIOpen` 만 찍히고 `MarkUIShown` 이 없어 계속 로그에 미완결 오픈이 남을 수 있으나, 이는 계속 노이즈일 뿐 동작 버그 아님 — 이번 범위 밖(원하면 실패 시 `CancelUIOpen` 류 혹 추가는 선택).

검증

- 정상: 팝업(예: UI_Info, ESC 메뉴) 열고 닫기 반복 — 매번 정상 오픈.
- 실패 재현: 존재하지 않는 키로 `ShowPopupUIAsync` 를 임시 호출 → null 반환 후 곧바로 정상 키 팝업이 다시 열리는지 확인(플래그 복구 증명). 확인 후 임시 코드 제거.

완료 체크리스트

- [] ① `LoadTextNoCacheAsync` 신설 + `DataManager` 전환 + `LoadAsyncNoCache` 제거 → 커밋
 - [] ② `scene→global` 승격 로직 추가 → 커밋
 - [] ③ `BaseScene.Clear()` 의 `Resource.Clear()` 제거 → 씬 왕복 assert 통과 → 커밋
 - [] ④ `ShowPopupUIAsync` try/finally → 실패 후 재오픈 검증 → 커밋
 - [] 공통: 씬 왕복 + 팝업 개폐 회귀 없음, 디버그 창 잔존 핸들 수 = Baseline v2와 동일
 - [] Refactor_Plan.md 결과 요약 표의 Phase 1 상태를 로, §6 재개 지점을 Phase 2로 갱신
- Phase 1은 "수치가 안 변해야 성공". 메모리/히치가 움직이면 오히려 의심할 것 — 이번 단계는 안전성만 손본다.